Event Management Module - Complete Guide

Table of Contents

1. Overview
2. Module Architecture
3. Getting Started
4. Database Schema
5. API Endpoints
6. Business Logic
7. How to Build and Run
8. Implementation Guide
9. Testing the Module
10. Troubleshooting

Overview

The **Event Management Module** is a comprehensive system for managing events, exhibitions, conferences, and festivals. It handles all aspects of event operations including:

- **Event Management**: Create and manage events with bilingual support (English/Arabic)
- **Visitor Management**: Track visitors, check-ins, tickets, and VIP handling
- **Booth & Tenant Management**: Manage exhibition booths, tenants, and contracts
- **Financial Operations**: Invoicing, payments, and financial tracking
- **Access Control**: Zones, gates, and permits management
- **Incident Management**: Track and resolve incidents during events

Key Features

- ☑ Bilingual support (English/Arabic)
- ☑ Real-time occupancy tracking
- ☑ QR code generation for tickets and visitors
- ☑ Financial management (invoices, payments, contracts)
- ☑ Multi-zone and multi-gate support
- ☑ Incident tracking and resolution
- ☑ PDF export for tickets and permits
- ☑ Excel import/export capabilities
- ☑ RESTful API with JWT authentication

Module Architecture

Layer Structure

```
EventManagement Module
│
├── ZR.Model.EventManagement/              # Data Models & DTOs
│   ├── Event.cs                           # Main event entity
│   ├── EventZone.cs                       # Zone management
│   ├── EventGate.cs                       # Gate/entrance management
│   ├── EventVisitor.cs                    # Visitor records
│   ├── EventTicket.cs                     # Ticket management
│   ├── EventPermit.cs                     # Access permits
│   ├── EventBooth.cs                      # Exhibition booths
│   ├── EventTenant.cs                     # Booth tenants
│   ├── EventContract.cs                   # Rental contracts
│   ├── EventInvoice.cs                    # Financial invoices
│   ├── EventPayment.cs                    # Payment records
│   ├── EventIncident.cs                   # Incident tracking
│   ├── EventType.cs                       # Lookup: Event types
│   ├── ZoneType.cs                        # Lookup: Zone types
│   ├── GateType.cs                        # Lookup: Gate types
│   ├── VisitorType.cs                     # Lookup: Visitor types
│   ├── TicketType.cs                      # Lookup: Ticket types
│   ├── PermitType.cs                      # Lookup: Permit types
│   ├── BoothType.cs                       # Lookup: Booth types
│   ├── IncidentType.cs                    # Lookup: Incident types
│   ├── PaymentMethod.cs                   # Lookup: Payment methods
│   └── Dto/
│       ├── EventManagementDtos.cs         # All DTOs
│       └── EventDashboardDto.cs           # Dashboard metrics
│
├── ZR.Repository.EventManagement/         # Data Access Layer
│   ├── EventRepository.cs
│   ├── EventZoneRepository.cs
│   ├── EventGateRepository.cs
│   ├── EventVisitorRepository.cs
│   ├── EventTicketRepository.cs
│   ├── EventPermitRepository.cs
│   ├── EventBoothRepository.cs
│   ├── EventTenantRepository.cs
│   ├── EventContractRepository.cs
│   ├── EventInvoiceRepository.cs
│   ├── EventPaymentRepository.cs
│   ├── EventIncidentRepository.cs
│   └── [Lookup table repositories...]
│
├── ZR.Service.EventManagement/            # Business Logic Layer
│   ├── IEventService/
│   │   └── IEventManagementServices.cs    # Service interfaces
│   ├── EventService.cs                    # Event operations
│   ├── EventZoneService.cs                # Zone management
│   ├── EventGateService.cs                # Gate management
│   ├── EventVisitorService.cs             # Visitor operations
│   ├── EventTicketService.cs              # Ticket operations
│   ├── EventPermitService.cs              # Permit management
│   ├── EventBoothTenantServices.cs        # Booth & tenant operations
```

```
│   ├── EventFinancialServices.cs          # Invoice & payment operations
│   ├── EventIncidentService.cs            # Incident management
│   ├── EventTypeService.cs                # Event type lookup
│   └── LookupTableServices.cs             # All lookup services
│
└── ZR.Admin.WebApi.Controllers.EventManagement/  # API Controllers
    ├── EventController.cs                 # Event CRUD API
    ├── EventZoneGateControllers.cs        # Zone & Gate APIs
    ├── EventVisitorTicketPermitControllers.cs  # Visitor, Ticket, Permit APIs
    ├── EventBoothTenantContractControllers.cs  # Booth, Tenant, Contract APIs
    ├── EventInvoicePaymentIncidentControllers.cs  # Financial & Incident APIs
    ├── LookupTableControllers.cs          # Lookup table APIs
    └── EventPdfExportController.cs        # PDF generation API
```

Database Tables

The module uses the following database tables (prefix: `event_`):

Main Tables:

- `event_events` - Events
- `event_zones` - Event zones
- `event_gates` - Entry/exit gates
- `event_visitors` - Visitor records
- `event_tickets` - Tickets
- `event_permits` - Access permits
- `event_booths` - Exhibition booths
- `event_tenants` - Booth tenants
- `event_contracts` - Rental contracts
- `event_invoices` - Invoices
- `event_payments` - Payments
- `event_incidents` - Incident reports

Lookup Tables:

- `event_types` - Event types (conference, exhibition, etc.)
- `event_zone_types` - Zone types
- `event_gate_types` - Gate types
- `event_visitor_types` - Visitor categories
- `event_ticket_types` - Ticket categories
- `event_permit_types` - Permit types
- `event_booth_types` - Booth categories
- `event_incident_types` - Incident categories
- `event_payment_methods` - Payment methods

Getting Started

Prerequisites

- .NET 8.0 SDK or later
- PostgreSQL 12+ or SQL Server 2019+
- Visual Studio 2022 or VS Code
- Docker (optional, for containerized deployment)
- Postman or similar API testing tool

Quick Start

1. Clone and Build

```
# Navigate to backend directory
cd F:\mahamBackendNET\backend

# Restore dependencies
dotnet restore ZRAdminMahamBackend.sln

# Build the solution
dotnet build ZRAdminMahamBackend.sln
```

2. Configure Database

Edit `ZR.Admin.WebApi/appsettings.json`:

```
{
  "dbConfigs": [
    {
      "ConfigId": "0",
      "DbType": 4,
      "ConnectionString":
"Host=srv792437.hstgr.cloud;Port=5433;Database=MahamDB;Username=your_user;Passwo
rd=your_pass",
      "IsAutoCloseConnection": true
    }
  ],
  "InitDb": false  // Set to true for first-time setup
}
```

3. Initialize Database (First Time Only)

Option A - Automatic Initialization:

1. Set `"InitDb": true` in `appsettings.json`
2. Run the application - tables will be created automatically

Option B - Manual Initialization via Code:

Add this to `Program.cs` before `app.Run()`:

```
// Initialize EventManagement tables
app.Environment.InitDbWithNamespaces("ZR.Model.EventManagement");
```

4. Run the Application

```
cd ZR.Admin.WebApi
dotnet run
```

The API will start on:

- HTTP: `http://localhost:8887`
- HTTPS: `https://localhost:8888`

5. Access API Documentation

Open your browser:

- **Scalar UI**: `http://localhost:8887/scalar` (Recommended)
- **Swagger UI**: `http://localhost:8887/swagger`

Database Schema

Core Entity: Event

```
CREATE TABLE event_events (
    event_id BIGINT PRIMARY KEY,
    event_code VARCHAR(50) UNIQUE,
    event_name VARCHAR(200) NOT NULL,
    event_name_ar VARCHAR(200),
    description TEXT,
    description_ar TEXT,
    event_type VARCHAR(50),
    status VARCHAR(50),  -- planned, active, completed, cancelled
    start_date TIMESTAMP NOT NULL,
    end_date TIMESTAMP NOT NULL,
    registration_start_date TIMESTAMP,
    registration_end_date TIMESTAMP,
    location_id BIGINT,
    location_name VARCHAR(200),
    address TEXT,
    city VARCHAR(100),
    country VARCHAR(100),
    venue VARCHAR(200),
    latitude DECIMAL(10,8),
    longitude DECIMAL(11,8),
    map_image_url VARCHAR(500),
```

```
    expected_visitors INT,
    max_capacity INT,
    current_occupancy INT DEFAULT 0,
    organizer VARCHAR(200),
    contact_email VARCHAR(100),
    contact_phone VARCHAR(50),
    website VARCHAR(200),
    logo_url VARCHAR(500),
    banner_image_url VARCHAR(500),
    is_public BOOLEAN DEFAULT true,
    requires_registration BOOLEAN DEFAULT true,
    is_paid_event BOOLEAN DEFAULT false,
    ticket_price DECIMAL(18,2),
    currency VARCHAR(10),
    event_manager_id BIGINT,
    tags TEXT,
    is_active BOOLEAN DEFAULT true,
    budget DECIMAL(18,2),
    actual_cost DECIMAL(18,2),
    total_area_sqm DECIMAL(18,2),
    expected_attendance DECIMAL(18,2),
    create_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    update_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Event Relationships

```
Event (1) ──→ (N) EventZone
Event (1) ──→ (N) EventGate
Event (1) ──→ (N) EventVisitor
Event (1) ──→ (N) EventTicket
Event (1) ──→ (N) EventPermit
Event (1) ──→ (N) EventBooth
Event (1) ──→ (N) EventContract
Event (1) ──→ (N) EventInvoice
Event (1) ──→ (N) EventPayment
Event (1) ──→ (N) EventIncident

EventBooth (N) ──→ (1) EventTenant
EventContract (N) ──→ (1) EventTenant
EventContract (N) ──→ (1) EventBooth
EventInvoice (N) ──→ (1) EventContract (optional)
EventPayment (N) ──→ (1) EventInvoice (optional)
EventVisitor (1) ──→ (N) EventTicket
```

API Endpoints

Base URL: `/event`

Event Management

Method	Endpoint	Description	Permission
GET	`/event/Event/list`	Get all events (paginated)	`event:list`
GET	`/event/Event/{eventId}`	Get event details	`event:query`
POST	`/event/Event`	Create new event	`event:add`
PUT	`/event/Event`	Update event	`event:edit`
POST	`/event/Event/delete/{ids}`	Delete events	`event:delete`
GET	`/event/Event/export`	Export events to Excel	`event:export`
POST	`/event/Event/importData`	Import events from Excel	`event:import`
GET	`/event/Event/importTemplate`	Download import template	(Public)
POST	`/event/Event/clean`	Clear all events (Admin only)	`event:delete`

Zone Management

Method	Endpoint	Description	Permission
GET	`/event/EventZone/list`	Get all zones	`eventzone:list`
GET	`/event/EventZone/{zoneId}`	Get zone details	`eventzone:query`
POST	`/event/EventZone`	Create zone	`eventzone:add`
PUT	`/event/EventZone`	Update zone	`eventzone:edit`
POST	`/event/EventZone/delete/{ids}`	Delete zones	`eventzone:delete`

Gate Management

Method	Endpoint	Description	Permission
GET	`/event/EventGate/list`	Get all gates	`eventgate:list`
GET	`/event/EventGate/{gateId}`	Get gate details	`eventgate:query`
POST	`/event/EventGate`	Create gate	`eventgate:add`
PUT	`/event/EventGate`	Update gate	`eventgate:edit`
POST	`/event/EventGate/delete/{ids}`	Delete gates	`eventgate:delete`

Visitor Management

Method	Endpoint	Description	Permission
GET	`/event/EventVisitor/list`	Get all visitors	`eventvisitor:list`

Method	Endpoint	Description	Permission
GET	`/event/EventVisitor/{visitorId}`	Get visitor details	`eventvisitor:query`
POST	`/event/EventVisitor`	Register visitor	`eventvisitor:add`
PUT	`/event/EventVisitor`	Update visitor	`eventvisitor:edit`
POST	`/event/EventVisitor/delete/{ids}`	Delete visitors	`eventvisitor:delete`
POST	`/event/EventVisitor/checkin/{visitorId}`	Check in visitor	`eventvisitor:checkin`
POST	`/event/EventVisitor/checkout/{visitorId}`	Check out visitor	`eventvisitor:checkout`

Ticket Management

Method	Endpoint	Description	Permission
GET	`/event/EventTicket/list`	Get all tickets	`eventticket:list`
GET	`/event/EventTicket/{ticketId}`	Get ticket details	`eventticket:query`
POST	`/event/EventTicket`	Issue ticket	`eventticket:add`
PUT	`/event/EventTicket`	Update ticket	`eventticket:edit`
POST	`/event/EventTicket/delete/{ids}`	Delete tickets	`eventticket:delete`
POST	`/event/EventTicket/validate/{ticketId}`	Validate ticket	`eventticket:validate`

Permit Management

Method	Endpoint	Description	Permission
GET	`/event/EventPermit/list`	Get all permits	`eventpermit:list`
GET	`/event/EventPermit/{permitId}`	Get permit details	`eventpermit:query`
POST	`/event/EventPermit`	Issue permit	`eventpermit:add`
PUT	`/event/EventPermit`	Update permit	`eventpermit:edit`
POST	`/event/EventPermit/delete/{ids}`	Delete permits	`eventpermit:delete`

Booth Management

Method	Endpoint	Description	Permission

Method	Endpoint	Description	Permission
GET	`/event/EventBooth/list`	Get all booths	`eventbooth:list`
GET	`/event/EventBooth/{boothId}`	Get booth details	`eventbooth:query`
POST	`/event/EventBooth`	Create booth	`eventbooth:add`
PUT	`/event/EventBooth`	Update booth	`eventbooth:edit`
POST	`/event/EventBooth/delete/{ids}`	Delete booths	`eventbooth:delete`

Tenant Management

Method	Endpoint	Description	Permission
GET	`/event/EventTenant/list`	Get all tenants	`eventtenant:list`
GET	`/event/EventTenant/{tenantId}`	Get tenant details	`eventtenant:query`
POST	`/event/EventTenant`	Create tenant	`eventtenant:add`
PUT	`/event/EventTenant`	Update tenant	`eventtenant:edit`
POST	`/event/EventTenant/delete/{ids}`	Delete tenants	`eventtenant:delete`

Contract Management

Method	Endpoint	Description	Permission
GET	`/event/EventContract/list`	Get all contracts	`eventcontract:list`
GET	`/event/EventContract/{contractId}`	Get contract details	`eventcontract:query`
POST	`/event/EventContract`	Create contract	`eventcontract:add`
PUT	`/event/EventContract`	Update contract	`eventcontract:edit`
POST	`/event/EventContract/delete/{ids}`	Delete contracts	`eventcontract:delete`

Invoice Management

Method	Endpoint	Description	Permission
GET	`/event/EventInvoice/list`	Get all invoices	`eventinvoice:list`
GET	`/event/EventInvoice/{invoiceId}`	Get invoice details	`eventinvoice:query`
POST	`/event/EventInvoice`	Create invoice	`eventinvoice:add`
PUT	`/event/EventInvoice`	Update invoice	`eventinvoice:edit`
POST	`/event/EventInvoice/delete/{ids}`	Delete invoices	`eventinvoice:delete`

Payment Management

Method	Endpoint	Description	Permission
GET	`/event/EventPayment/list`	Get all payments	`eventpayment:list`
GET	`/event/EventPayment/{paymentId}`	Get payment details	`eventpayment:query`
POST	`/event/EventPayment`	Record payment	`eventpayment:add`
PUT	`/event/EventPayment`	Update payment	`eventpayment:edit`
POST	`/event/EventPayment/delete/{ids}`	Delete payments	`eventpayment:delete`

Incident Management

Method	Endpoint	Description	Permission
GET	`/event/EventIncident/list`	Get all incidents	`eventincident:list`
GET	`/event/EventIncident/{incidentId}`	Get incident details	`eventincident:query`
POST	`/event/EventIncident`	Create incident	`eventincident:add`
PUT	`/event/EventIncident`	Update incident	`eventincident:edit`
POST	`/event/EventIncident/delete/{ids}`	Delete incidents	`eventincident:delete`
POST	`/event/EventIncident/resolve/{incidentId}`	Mark incident as resolved	`eventincident:resolve`

Lookup Tables

Method	Endpoint	Description
GET	`/event/EventType/list`	Get event types
GET	`/event/ZoneType/list`	Get zone types
GET	`/event/GateType/list`	Get gate types
GET	`/event/VisitorType/list`	Get visitor types
GET	`/event/TicketType/list`	Get ticket types
GET	`/event/PermitType/list`	Get permit types
GET	`/event/BoothType/list`	Get booth types
GET	`/event/IncidentType/list`	Get incident types
GET	`/event/PaymentMethod/list`	Get payment methods

PDF Export

Method	Endpoint	Description	Permission
GET	`/event/EventPdf/ticket/{ticketId}`	Generate ticket PDF	`eventticket:export`
GET	`/event/EventPdf/permit/{permitId}`	Generate permit PDF	`eventpermit:export`
GET	`/event/EventPdf/invoice/{invoiceId}`	Generate invoice PDF	`eventinvoice:export`
GET	`/event/EventPdf/contract/{contractId}`	Generate contract PDF	`eventcontract:export`

Business Logic

Event Lifecycle

1. **Planned** → Event is created and being planned
2. **Active** → Event is currently running
3. **Completed** → Event has finished
4. **Cancelled** → Event was cancelled

Visitor Check-in Flow

```
1. Visitor registers → Status: "registered"
2. Ticket is issued → Status: "issued"
3. Visitor arrives and checks in → Status: "checked_in"
4. Visitor leaves and checks out → Status: "checked_out"
```

Booth Rental Flow

```
1. Create Booth → Status: "available"
2. Tenant registers
3. Create Contract → Booth Status: "reserved"
4. Generate Invoice
5. Record Payment
6. Contract signed → Booth Status: "occupied"
7. Event ends → Booth Status: "available"
```

Financial Flow

```
1. Contract created → TotalAmount calculated
2. Invoice generated → SubtotalAmount + VATAmount = TotalAmount
```

```
3. Payment recorded → PaidAmount increments
4. Invoice Status:
   - "pending" (PaidAmount = 0)
   - "partial" (0 < PaidAmount < TotalAmount)
   - "paid" (PaidAmount >= TotalAmount)
5. Payment methods: cash, card, bank_transfer, online
```

Incident Management Flow

```
1. Incident reported → Status: "reported"
2. Assigned to staff → Status: "assigned"
3. Staff responds → Status: "in_progress"
4. Issue resolved → Status: "resolved"
5. Incident closed → Status: "closed"
```

How to Build and Run

Local Development

Method 1: Visual Studio

1. Open `ZRAdminMahamBackend.sln` in Visual Studio 2022
2. Set `ZR.Admin.WebApi` as startup project
3. Press F5 or click Run

Method 2: Command Line

```
# Navigate to solution directory
cd F:\mahamBackendNET\backend

# Restore packages
dotnet restore

# Build solution
dotnet build ZRAdminMahamBackend.sln --configuration Release

# Run the API
cd ZR.Admin.WebApi
dotnet run --configuration Release
```

Method 3: Watch Mode (Auto-reload)

```
cd F:\mahamBackendNET\backend\ZR.Admin.WebApi
dotnet watch run
```

Docker Deployment

Build Docker Image

```
cd F:\mahamBackendNET\backend

# Build image
docker build -t zr-admin-api:latest -f ZR.Admin.WebApi/Dockerfile .

# Run container
docker run -d -p 8887:8887 -p 8888:8888 \
  -e ASPNETCORE_ENVIRONMENT=Production \
  -e ConnectionStrings__DefaultConnection="Host=your-
db;Port=5432;Database=MahamDB;..." \
  --name zr-admin-api \
  zr-admin-api:latest
```

Docker Compose

```
# Development
docker-compose -f docker-compose.yml -f docker-compose.override.yml up -d

# Production
docker-compose -f docker-compose.yml -f docker-compose.prod.yml up -d

# View logs
docker-compose logs -f zr-admin-api

# Stop services
docker-compose down
```

Environment Configuration

Development (appsettings.Development.json)

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Debug",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "InitDb": true,
  "DemoMode": false
}
```

Production (appsettings.Production.json)

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "InitDb": false,
  "DemoMode": false
}
```

Implementation Guide

Step-by-Step: Adding Event Management to Your Project

Step 1: Database Setup

```
# Run the application with InitDb=true to auto-create tables
cd ZR.Admin.WebApi
dotnet run
```

Or manually initialize in `Program.cs`:

```
// Add before app.Run()
using ZR.ServiceCore.SqlSugar;

if (args.Contains("--init-eventmanagement"))
{
    app.Environment.InitDbWithNamespaces("ZR.Model.EventManagement");
    Console.WriteLine("EventManagement tables initialized!");
}
```

Run with:

```
dotnet run -- --init-eventmanagement
```

Step 2: Seed Lookup Data

Create a seed data script or use the API to populate lookup tables:

```sql
-- Event Types
INSERT INTO event_types (type_code, type_name, type_name_ar, is_active) VALUES
('CONFERENCE', 'Conference', 'مؤتمر', true),
('EXHIBITION', 'Exhibition', 'معرض', true),
('FESTIVAL', 'Festival', 'مهرجان', true),
('CONCERT', 'Concert', 'حفل موسيقي', true),
('WORKSHOP', 'Workshop', 'ورشة عمل', true);

-- Zone Types
INSERT INTO event_zone_types (type_code, type_name, type_name_ar, is_active)
VALUES
('MAIN_HALL', 'Main Hall', 'القاعة الرئيسية', true),
('VIP_LOUNGE', 'VIP Lounge', 'صالة كبار الشخصيات', true),
('EXHIBITION_AREA', 'Exhibition Area', 'منطقة المعرض', true),
('PARKING', 'Parking', 'موقف السيارات', true),
('FOOD_COURT', 'Food Court', 'منطقة الطعام', true);

-- Gate Types
INSERT INTO event_gate_types (type_code, type_name, type_name_ar, direction,
is_active) VALUES
('MAIN_ENTRANCE', 'Main Entrance', 'المدخل الرئيسي', 'entry', true),
('VIP_ENTRANCE', 'VIP Entrance', 'مدخل كبار الشخصيات', 'entry', true),
('EMERGENCY_EXIT', 'Emergency Exit', 'مخرج الطوارئ', 'exit', true),
('SIDE_ENTRANCE', 'Side Entrance', 'المدخل الجانبي', 'entry', true);

-- Visitor Types
INSERT INTO event_visitor_types (type_code, type_name, type_name_ar, is_active)
VALUES
('GENERAL', 'General Public', 'الجمهور العام', true),
('VIP', 'VIP', 'كبار الشخصيات', true),
('PRESS', 'Press/Media', 'الصحافة والإعلام', true),
('EXHIBITOR', 'Exhibitor', 'عارض', true),
('STAFF', 'Staff', 'طاقم العمل', true);

-- Ticket Types
INSERT INTO event_ticket_types (type_code, type_name, type_name_ar, is_active)
VALUES
('STANDARD', 'Standard', 'عادي', true),
('VIP', 'VIP', 'كبار الشخصيات', true),
('EARLY_BIRD', 'Early Bird', 'حجز مبكر', true),
('GROUP', 'Group', 'مجموعة', true),
('STUDENT', 'Student', 'طالب', true);

-- Booth Types
INSERT INTO event_booth_types (type_code, type_name, type_name_ar, is_active)
VALUES
('STANDARD', 'Standard', 'عادي', true),
('PREMIUM', 'Premium', 'مميز', true),
('CORNER', 'Corner', 'ركن', true),
('ISLAND', 'Island', 'جزيرة', true);

-- Incident Types
```

```sql
INSERT INTO event_incident_types (type_code, type_name, type_name_ar, severity,
is_active) VALUES
('MEDICAL', 'Medical Emergency', 'حالة طبية طارئة', 'critical', true),
('SECURITY', 'Security Issue', 'مشكلة أمنية', 'high', true),
('TECHNICAL', 'Technical Issue', 'مشكلة تقنية', 'medium', true),
('COMPLAINT', 'Complaint', 'شكوى', 'low', true);

-- Payment Methods
INSERT INTO event_payment_methods (method_code, method_name, method_name_ar,
is_active) VALUES
('CASH', 'Cash', 'نقدي', true),
('CARD', 'Credit/Debit Card', 'بطاقة ائتمان', true),
('BANK_TRANSFER', 'Bank Transfer', 'تحويل بنكي', true),
('ONLINE', 'Online Payment', 'دفع إلكتروني', true);
```

Step 3: Configure Permissions

Add permissions to your system's menu configuration:

```json
{
  "menuName": "Event Management",
  "path": "/event",
  "permissions": [
    "event:list",
    "event:query",
    "event:add",
    "event:edit",
    "event:delete",
    "event:export",
    "event:import"
  ],
  "children": [
    {
      "menuName": "Events",
      "path": "/event/events",
      "permissions": ["event:list", "event:add", "event:edit", "event:delete"]
    },
    {
      "menuName": "Visitors",
      "path": "/event/visitors",
      "permissions": ["eventvisitor:list", "eventvisitor:add",
"eventvisitor:checkin"]
    },
    {
      "menuName": "Booths",
      "path": "/event/booths",
      "permissions": ["eventbooth:list", "eventbooth:add", "eventbooth:edit"]
    }
  ]
}
```

Step 4: Test the APIs

Use Postman, curl, or the built-in Scalar UI to test:

Create an Event:

```
curl -X POST "http://localhost:8887/event/Event" \
  -H "Authorization: Bearer YOUR_JWT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "eventCode": "CONF2025",
    "eventName": "Tech Conference 2025",
    "eventNameAr": "مؤتمر التقنية 2025",
    "eventType": "CONFERENCE",
    "status": "planned",
    "startDate": "2025-03-15T09:00:00",
    "endDate": "2025-03-17T18:00:00",
    "locationName": "Riyadh Convention Center",
    "city": "Riyadh",
    "country": "Saudi Arabia",
    "maxCapacity": 5000,
    "isActive": true
  }'
```

Get Events List:

```
curl -X GET "http://localhost:8887/event/Event/list?pageNum=1&pageSize=10" \
  -H "Authorization: Bearer YOUR_JWT_TOKEN"
```

Register a Visitor:

```
curl -X POST "http://localhost:8887/event/EventVisitor" \
  -H "Authorization: Bearer YOUR_JWT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "eventId": 123456789,
    "visitorType": "GENERAL",
    "firstName": "Ahmed",
    "lastName": "Ali",
    "firstNameAr": "أحمد",
    "lastNameAr": "علي",
    "email": "ahmed.ali@example.com",
    "phone": "+966501234567",
    "status": "registered",
    "isVIP": false
  }'
```

Testing the Module

Manual Testing Checklist

Event Management

- [ ] Create a new event
- [ ] Update event details
- [ ] List all events with pagination
- [ ] Filter events by date range
- [ ] Export events to Excel
- [ ] Import events from Excel
- [ ] Delete an event
- [ ] View event details

Visitor Management

- [ ] Register a new visitor
- [ ] Check in a visitor
- [ ] Check out a visitor
- [ ] List visitors by event
- [ ] Filter visitors by type (VIP, General, etc.)
- [ ] Generate visitor badge (QR code)
- [ ] Export visitor list to Excel

Ticket Management

- [ ] Issue a ticket to a visitor
- [ ] Validate a ticket (QR scan)
- [ ] List all tickets for an event
- [ ] Filter tickets by status
- [ ] Generate ticket PDF
- [ ] Transfer ticket to another visitor

Booth & Tenant Management

- [ ] Create booths for an event
- [ ] Register a tenant company
- [ ] Create a rental contract
- [ ] Assign booth to tenant
- [ ] Generate contract PDF
- [ ] Update booth rental status

Financial Operations

- [ ] Create an invoice from a contract
- [ ] Record a payment
- [ ] Generate invoice PDF

- ☐ View payment history
- ☐ Calculate VAT
- ☐ Track outstanding balances

Incident Management

- ☐ Report a new incident
- ☐ Assign incident to staff
- ☐ Update incident status
- ☐ Resolve an incident
- ☐ View incident history
- ☐ Filter incidents by severity

API Testing with Postman

1. Import the Swagger JSON into Postman
2. Set up environment variables:
   - `baseUrl`: `http://localhost:8887`
   - `token`: Your JWT token
3. Run the collection tests

Performance Testing

```
# Install Apache Bench
apt-get install apache2-utils

# Test event list endpoint (100 requests, 10 concurrent)
ab -n 100 -c 10 -H "Authorization: Bearer YOUR_TOKEN" \
  http://localhost:8887/event/Event/list
```

Troubleshooting

Common Issues

1. Database Tables Not Created

Problem: Tables are not being created automatically.

Solution:

- Set `"InitDb": true` in `appsettings.json`
- Ensure database connection string is correct
- Check logs for SqlSugar errors
- Manually initialize: `app.Environment.InitDbWithNamespaces("ZR.Model.EventManagement")`

2. Permission Denied Errors

Problem: API returns 403 Forbidden.

Solution:

- Ensure user role has required permissions
- Check permission strings match exactly (e.g., `event:list`)
- Verify JWT token is valid and not expired
- Check user is assigned to a role with EventManagement permissions

3. QR Code Not Generating

Problem: Ticket/visitor QR codes are empty.

Solution:

- Ensure QR code generation library is installed
- Check `QRCode` field in DTOs
- Verify QR service is properly injected

4. PDF Export Fails

Problem: PDF generation returns 500 error.

Solution:

- Check Puppeteer/wkhtmltopdf is installed in Docker container
- Verify font files are available (for Arabic text)
- Check `ZR.Pdf` service configuration
- View detailed error in logs

5. Excel Import Fails

Problem: Import returns errors or no data inserted.

Solution:

- Use the import template from `/event/Event/importTemplate`
- Ensure Excel file format matches template
- Check required fields are filled (EventName, EventCode)
- Verify data types match (dates, numbers)

6. Slow Query Performance

Problem: List endpoints are slow with large datasets.

Solution:

- Add database indexes on frequently queried fields
- Use pagination (limit page size to 50-100 records)
- Optimize query predicates
- Enable SQL query logging to identify slow queries

```
-- Recommended indexes
CREATE INDEX idx_event_start_date ON event_events(start_date);
CREATE INDEX idx_event_status ON event_events(status);
CREATE INDEX idx_visitor_event_id ON event_visitors(event_id);
CREATE INDEX idx_ticket_visitor_id ON event_tickets(visitor_id);
CREATE INDEX idx_booth_event_id ON event_booths(event_id);
```

Debug Mode

Enable detailed logging in `appsettings.Development.json`:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Debug",
      "ZR.Service.EventManagement": "Trace",
      "SqlSugar": "Debug"
    }
  }
}
```

Check Database Connection

```
# Test PostgreSQL connection
psql -h srv792437.hstgr.cloud -p 5433 -U your_user -d MahamDB -c "SELECT 1;"

# List EventManagement tables
psql -h srv792437.hstgr.cloud -p 5433 -U your_user -d MahamDB -c "\dt event_*"
```

View Application Logs

```
# Development logs
tail -f ZR.Admin.WebApi/logs/app-{date}.log

# Docker container logs
docker logs -f zr-admin-api

# Docker Compose logs
docker-compose logs -f
```

Additional Resources

Related Documentation

- CLAUDE.md - Project overview and conventions
- DEPLOYMENT_README.md - Deployment guide
- Swagger/Scalar UI - Interactive API docs

Code Generator

The EventManagement module was built using the built-in code generator. To generate similar modules:

1. Configure `codeGen.json`
2. Design your database tables
3. Run code generator from admin UI
4. Generated files include models, DTOs, repositories, services, and controllers

Support

- GitHub Issues: Project Issues
- Email: support@example.com

Last Updated: 2025-10-14
Version: 1.0.0
Module Status: Production-Ready ☑